

Behavior Tree 개요

Behavior Tree

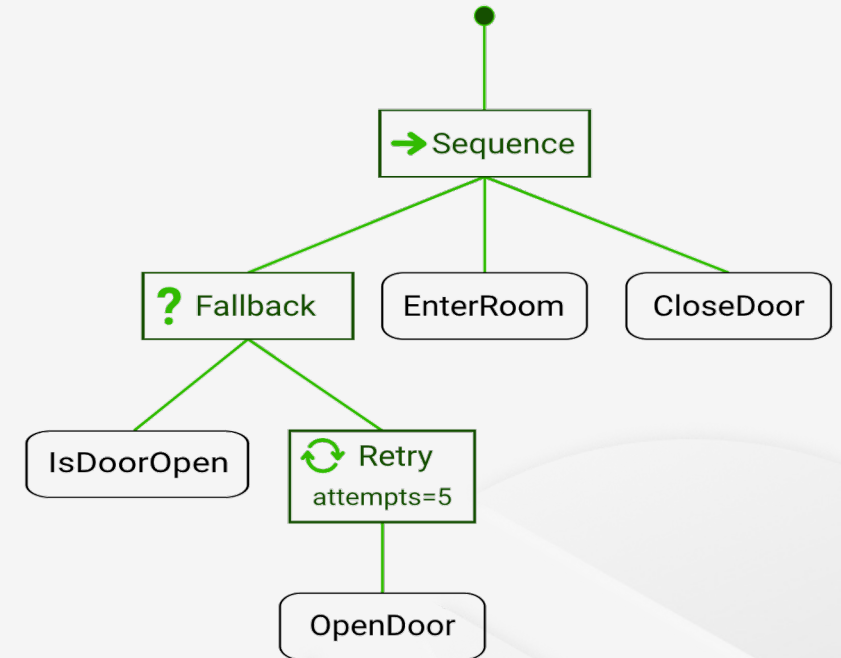
Behavior Tree란?

- Behavior Tree는 트리 구조

- ✓ 루트 노드에서 시작하여 성공 또는 실패라는 최종 상태에 도달할 때까지 특정 순서로 순회하도록 설계됨

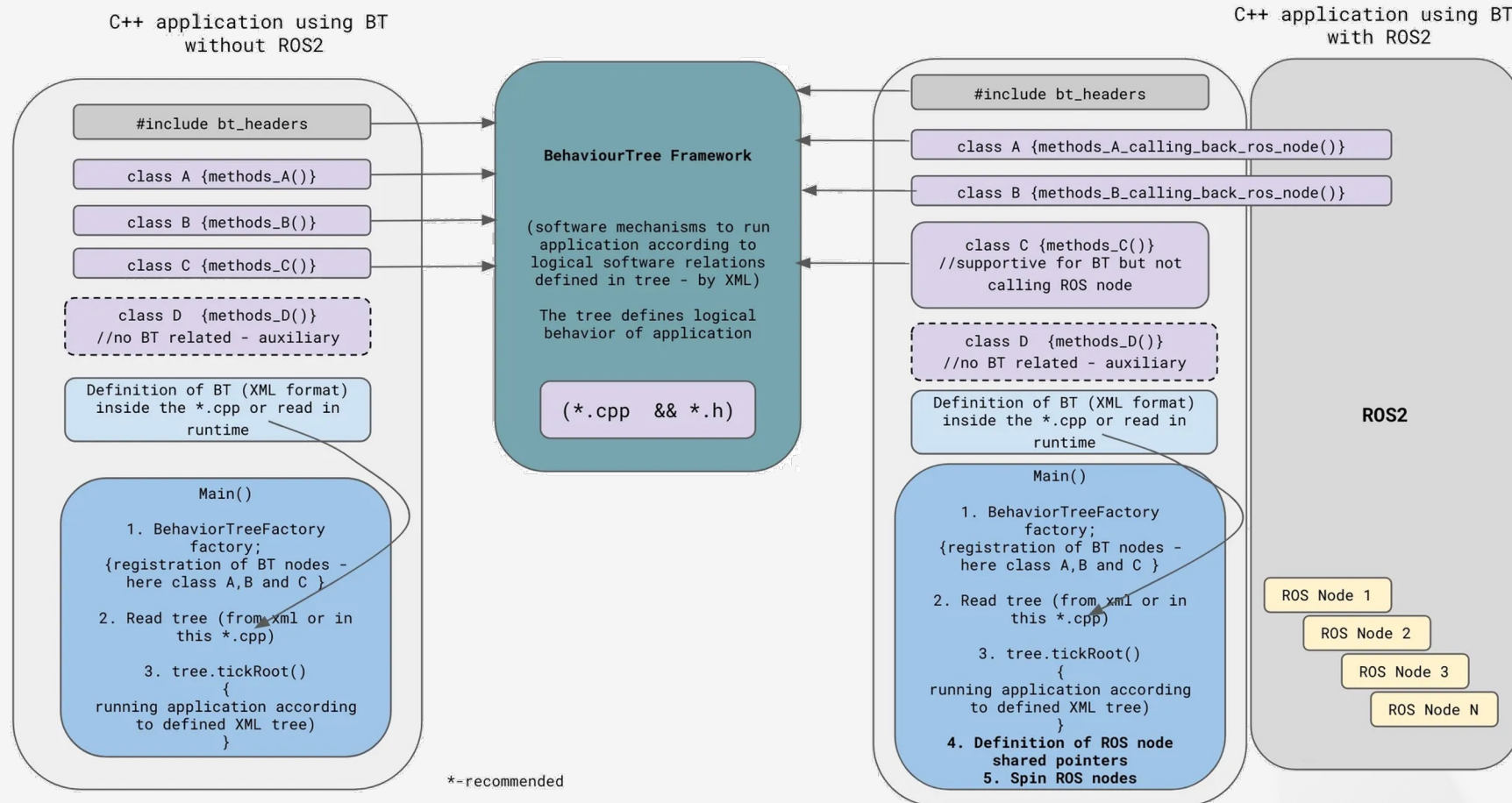
- 리프 노드(leaf node)는 실행 가능한 행동

- ✓ 각 리프는 간단한 확인이든 복잡한 동작이든 무언가를 수행하고, 상태(성공, 실패 또는 실행 중)를 출력
- ✓ 다시 말해, 리프 노드는 Behavior Tree를 특정 응용 프로그램을 위한 하위 수준 코드와 연결하는 지점



BT 활용을 위한 Software Architecture

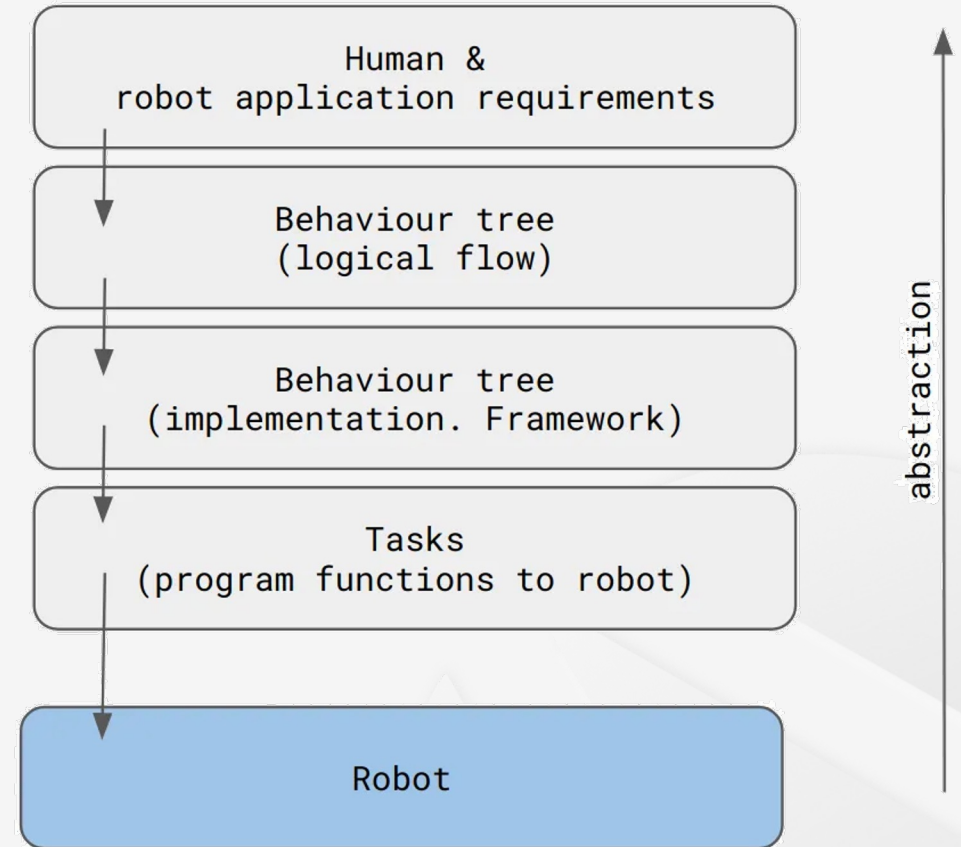
Simplified software architecture*



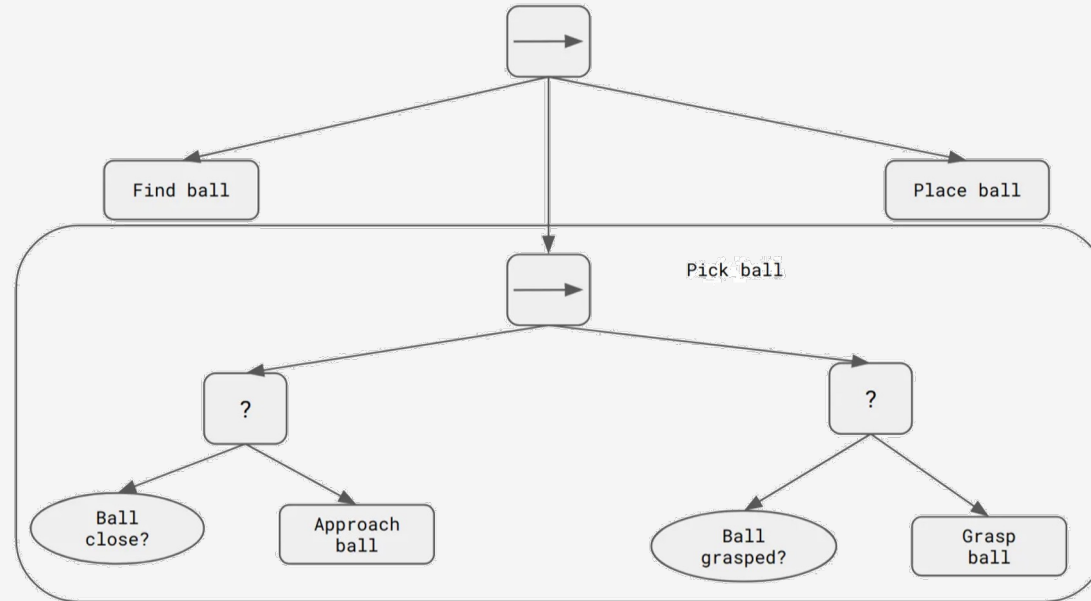
Concept

- 인간은 이 스택의 최상위에 있으며, Behavior Trees 개념을 사용하여 필요한 로봇 애플리케이션의 논리적 추론을 설계
- 그 다음 인간은 Behavior Trees를 사용하여 로봇 작업 간의 논리적 연결을 모델링
- Behavior Trees의 구현할 때에는 XML 파일로 정의하며, 이 파일의 흐름대로 통해 동작을 수행
- Task 하나하나를 구현할 때에는 BehaviourTree.CPP 프레임워크를 사용하며, 이는 계층과 Behavior Trees 구축을 가능하도록 함

BT abstraction stack



행동 예시

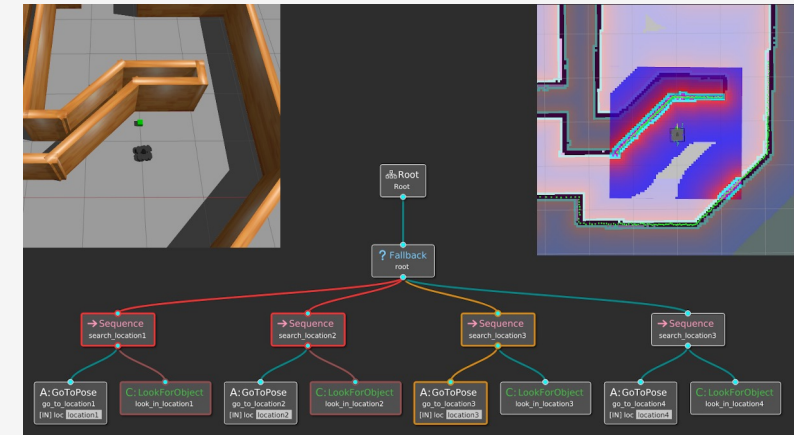


- "→" 기호: 논리적 AND 연산(Sequence)
 - ✓ 이 Sequence 블록 아래의 모든 동작이 성공해야 "트리의 분기"가 성공
- "?" 기호는 논리적 OR(Fallback)
 - ✓ 이 Fallback 블록 아래의 하나의 동작만 성공하면 (그 트리 분기의) 동작 실행이 종료(성공)

Behavior Tree **관련** Tool

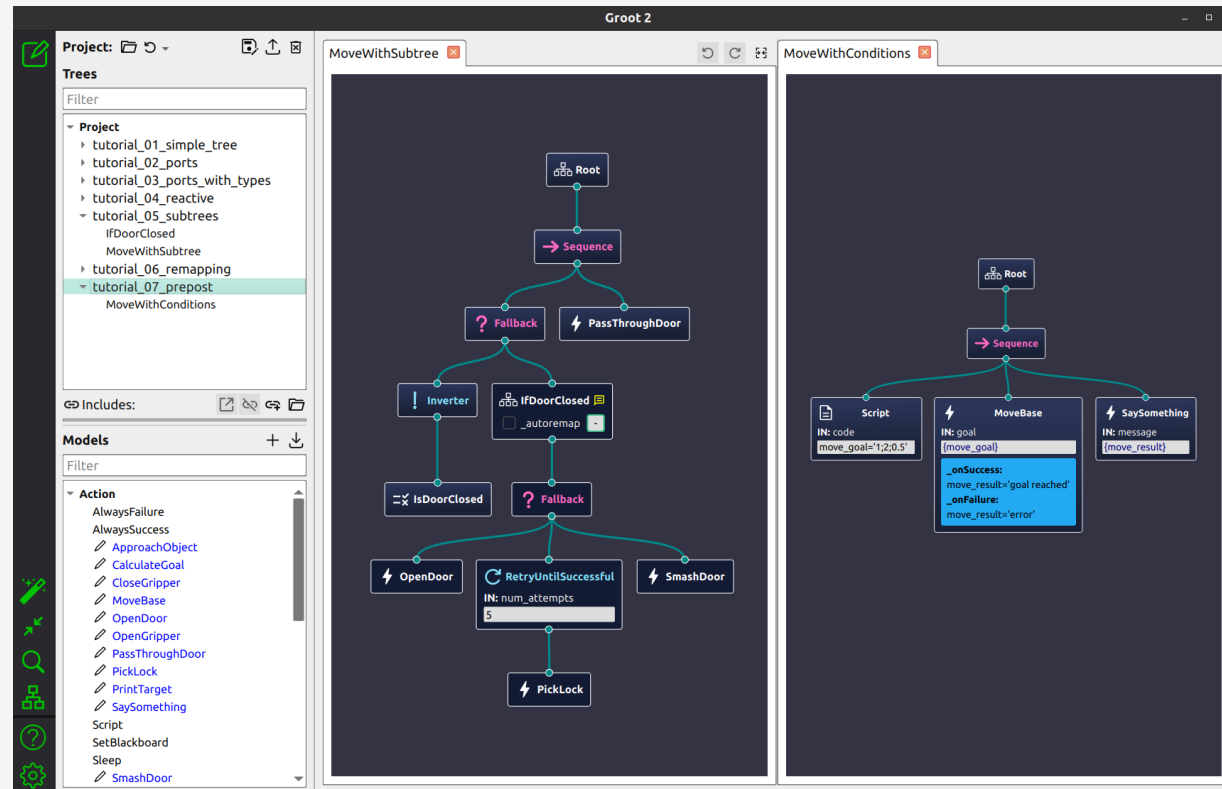
BehaviorTree.CPP

- C++로 Behavior Tree를 작성하고 실행할 수 있는 라이브러리
- 주요 특징
 - ✓ 노드 타입 등록, 트리 생성 및 실행을 위한 간단한 API를 제공
 - ✓ SyncActionNode 및 AsyncActionNode 클래스를 통해 동기 및 비동기 행동을 정의
 - ✓ 사용자 정의 노드를 쉽게 정의하고 등록할 수 있음 (조건 노드, 액션 노드, 컨트롤러 노드(시퀀스, 셀렉터 등) 등을 지원)
 - ✓ XML 파일을 사용하여 Behavior Tree 구조를 정의할 수 있으며, 이를 통해 트리를 쉽게 저장하고 로드할 수 있음



Groot

- GUI를 통해 Behavior Tree의 상태를 시각화할 수 있는 도구



Groot

- Nav2 지원 중단 (Humble 기준)
 - ✓ 기술적 한계
 - ✓ 실용성 문제

Dropping Support for Live Groot Monitoring of Nav2

- <https://github.com/ros-planning/navigation2/pull/2696>

It was a great feature idea but never quite panned out, especially after we introduced multiple navigator types in the BT Navigator server. The issue we run into primarily is that Zero-MQ prevents users from producing multiple logger types in the same process. Since BT nav has multiple servers, the swapping between them for viewing has never had a clean hand off causing folks to file tickets or have nasty logs appear or ZMQ crashes in the background. The BT.CPP client for this doesn't allow us to have a clean shutdown process so we're left with hoping that ZMQ properly handles the situation, which it rarely does. Further, Groot only supports visualizing one type of tree at a time so for applications often switching between navigator types, its not possible to use a single groot client, causing great frustration.

So, what I propose here is to remove live monitoring of the BT from Nav2. **We can still use Groot to modify, visualize, and generally work with behavior trees**, the only thing being removed is to live view the executing behavior tree as Nav2 is currently executing it (it used to light up the boxes of the current nodes). This was of dubious value anyhow, since the tree ticks so fast its difficult to visualize and get meaningful insights into things as the system is moving so quickly.

BehaviorTree.CPP + Groot



Thank You